



**INSTITUTE FOR ADVANCED
COMPUTING AND SOFTWARE
DEVELOPMENT, AKURDI, PUNE**

“LogiChain”

PG-DAC August 2025

Submitted By:

Group No: 86

Roll No.

258151

Name of Student

Kamlesh Shrikant Kasambe

Mrs. Manjari Deshpande

Project Guide

Mr. Anil Sharma

Centre Coordinator

ABSTRACT

LogiChain is a real-time shipment and inventory tracking system designed to enhance supply chain visibility and operational efficiency in e-commerce environments. The system enables businesses to monitor inventory levels, manage warehouses, process orders, and track shipments from origin to delivery. By providing end-to-end visibility across the supply chain, LogiChain helps reduce delays, prevent inventory inconsistencies, and improve overall customer satisfaction.

LogiChain is a scalable and modular application developed using Spring Boot for the backend and React with React for the frontend, with MySQL used for reliable data persistence. The system follows a microservices-based architecture to manage core functionalities such as order fulfillment, inventory and warehouse management, and shipment tracking.

ACKNOWLEDGEMENT

I take this occasion to thank God, almighty for blessing us with his grace and taking our endeavor to a successful culmination. I extend my heartfelt thanks to our esteemed guide, **Mrs. Manjiri Deshpande** for providing me with the right guidance and advice at the crucial juncture and showing me the right way. I sincerely thank our respected Centre Co-Ordinator, Mr. Anil Sharma, for allowing us to use the available facilities. I would also like to thank the other faculty members at this occasion. Last but not least, I would like to thank my friends and family for the support and encouragement they have given me during our work.

Kamlesh Shrikant Kasambe (250841220076)

Table of Contents

Sr.No	Description	Page No.
1	Introduction	1
2	SRS	2
3	Diagrams	10
3.1	ER Diagram	10
3.2	Use Case Diagram	11
3.3	Data Flow Diagram	12
3.4	Activity Diagram	13
3.5	Class Diagram	16
3.6	Component Diagram	18
3.7	Sequence Diagram	17
4	Database Design	19
5	Snapshots	24
6	Conclusion	27
7	References	28

1. INTRODUCTION

In the rapidly growing e-commerce industry, efficient supply chain management has become a critical factor for business success. Customers increasingly expect timely deliveries, accurate order tracking, and real-time updates, while businesses must maintain precise control over inventory, warehouses, and shipments. Traditional logistics systems often lack real-time visibility, auditability, and scalability, leading to delays, stock inconsistencies, and reduced customer satisfaction. To overcome these challenges, modern digital solutions that integrate shipment tracking and inventory management are essential.

LogiChain is a real-time shipment and inventory tracking system designed to enhance supply chain transparency and operational efficiency for e-commerce platforms. The system provides end-to-end visibility of the logistics process, tracking products from warehouses through shipments and ultimately to customers. By integrating inventory management with shipment tracking and order fulfilment, LogiChain ensures accurate stock control, timely deliveries, and improved coordination across the supply chain.

The application is built using a microservices architecture, enabling modular development, scalability, and independent service management. Core services include the Order Fulfilment Service, Inventory & Warehouse Service, and Tracking & Notification Service, each responsible for a specific domain of the supply chain. This separation of concerns improves system reliability and makes the platform adaptable to future business and technological changes.

LogiChain leverages Spring Boot, Spring Data JPA, and Hibernate for building a robust and scalable backend, while MySQL is used for persistent data storage. Real-time shipment updates are supported through WebSocket-based communication, allowing live tracking of shipment status and location. On the frontend, React with TypeScript is used to deliver an interactive and user-friendly interface, complemented by dashboard analytics for monitoring shipment and inventory metrics.

Security is a key aspect of the LogiChain system. It implements role-based authentication and authorization, ensuring that users can access only the functionalities relevant to their roles, such as administrators, warehouse managers, customer support staff, and customers. Secure password handling, controlled access to data, and audit-ready shipment events contribute to the reliability and trustworthiness of the platform.

Overall, LogiChain provides a comprehensive solution for real-time logistics and inventory management in e-commerce environments. By combining modern technologies, secure system design, and real-time data visibility, the system lays a strong foundation for efficient supply chain operations, improved customer satisfaction, and future scalability.

2. SRS

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for LogiChain, a centralized Warehouse and Supply Chain Management System. It serves as a formal contract between stakeholders and the development team, defining system behavior, role-based access constraints, and data integrity rules to ensure consistent and secure system implementation.

1.2 Scope

LogiChain is a full-stack, role-driven ERP platform designed to manage e-commerce middle-mile and last-mile operations.

The system automates the complete lifecycle of logistics operations, including product management, warehouse handling, inventory synchronization, order processing, shipment tracking, and returns management.

The system supports the following user roles:

- Admin
- Product Manager
- Warehouse Manager
- Customer Support
- Customer

Each role interacts with the system through strictly enforced Role-Based Access Control (RBAC) policies.

1.3 Definitions, Acronyms, and Abbreviations

- **SRS** – Software Requirements Specification
- **RBAC** – Role-Based Access Control
- **JWT** – JSON Web Token
- **OAuth** – Open Authorization Protocol
- **API** – Application Programming Interface
- **ERP** – Enterprise Resource Planning

1.4 References

- IEEE 29148-2018 Software Requirements Specification
- Spring Boot Documentation
- ReactJS Documentation
- MySQL Documentation

1.5 Document Overview

This document describes the system architecture, external interfaces, functional requirements, non-functional requirements, constraints, and future enhancements of LogiChain.

2. Overall Description

2.1 Product Perspective

LogiChain is a web-based enterprise system built using a microservice-oriented REST architecture.

The backend is implemented using Spring Boot, while the frontend uses ReactJS for dynamic, role-based user interfaces.

The system integrates authentication, authorization, and real-time data synchronization across modules.

2.2 Product Functions

The major functions of the system include:

- Secure user authentication and authorization
- Product and inventory management
- Warehouse and stock tracking
- Order processing and shipment handling

2.3 User Classes and Characteristics

Role	Responsibility	Authority Level
Admin	System governance, User Lifecycle Management (ULM).	Level 4 (Full)
Product Manager	Product metadata, catalog accuracy, and global inventory visibility.	Level 3 (Creation)
Warehouse Manager	Physical stock accuracy, shipment logistics, and warehouse-specific	Level 3 (Logistics)
Customer Support	Post-purchase mediation and shipment visibility.	Level 2 (Update)
Customer	End-user consumption and personal order tracking.	Level 1 (Restricted)

Table-1: User Classes and Characteristics

2.4 Operating Environment

- Backend: Java Spring Boot
- Frontend: ReactJS with Tailwind CSS
- Database: MySQL
- Server: Apache Tomcat
- OS: Windows / Linux

2.5 Constraints

- RBAC must be strictly enforced
- Secure authentication is mandatory
- Data consistency must be maintained across modules

3. External Interface Requirements

3.1 For Development, software's used are:

- **Operating System:**
Windows 10 / Windows 11 / Linux
- **Platform:**
Web Application
- **Technology:**
Spring Boot, ReactJS
- **Language:**
Java, JavaScript
- **Backend:**
Java (Spring Boot)
- **Editor:**
Visual Studio Code (VS Code), Spring Tool Suite (STS)
- **For Development:**
Spring Boot (v3.5.7)
ReactJS
MySQL
Git and GitHub
Postman
JUnit (v6.0.1)
- **For Design:**
HTML
CSS
JavaScript
Tailwind CSS / Bootstrap

3.2 For Development, Hardware's used are:

- **Processor:**
Intel i5 (11th Generation or equivalent)
- **Hard Disk:**
512 GB SSD
- **RAM:**
16 GB

3.3 Communication Interface

- HTTPS protocol
- JSON-based API communication
- Axios for frontend HTTP requests

4. Functional Requirements (Categorized by Module)

4.1 User Authentication and Authorization

- The system shall support JWT-based authentication for internal users (Admin, Product Manager, Warehouse Manager, Customer Support).
- The system shall support OAuth-based authentication for customers.
- The system shall implement Role-Based Access Control (RBAC) to restrict access based on assigned roles.
- The system shall ensure users can only access APIs and UI components authorized for their role.
- The system shall securely manage session tokens and credentials.

4.2 Product Management

- The system shall allow Product Managers to create, update, and delete product records.
- The system shall store product attributes such as SKU, weight, dimensions, and pricing.
- The system shall ensure product data consistency across warehouses.

4.3 Inventory Management

- The system shall track inventory quantities in real time.
- The system shall automatically update stock levels based on order placement and shipment confirmation.
- The system shall prevent inventory underflow conditions

4.4 Warehouse Management

- The system shall allow management of warehouse locations and capacities.
- The system shall track product-to-warehouse allocation.
- The system shall monitor warehouse utilization.

4.5 Order Management

- The system shall allow customers to place orders.
- The system shall automatically initiate shipment creation upon order confirmation.
- The system shall track order lifecycle states.

4.6 Shipment Management

- The system shall generate shipment records with tracking numbers.
- The system shall update shipment status (dispatched, in transit, delivered).
- The system shall maintain estimated delivery timelines.

4.7 Return and Support Management

- The system shall allow customers to initiate return requests.
- The system shall allow Customer Support to process returns and exchanges.
- The system shall update inventory after successful returns.

5. Non-Functional Requirements

5.1 Performance Requirements

- The system shall support concurrent users without performance degradation.
- API response times shall be within acceptable limits.

5.2 Security Requirements

- All endpoints shall be secured using Spring Security.
- Passwords shall be encrypted.
- Tokens shall have expiration and refresh mechanisms.

5.3 Reliability and Availability

- The system shall handle failures gracefully.
- Logging and monitoring shall be enabled.

5.4 Scalability

- The system shall support horizontal scaling.
- Modular architecture shall allow future extensions.

6. Role-Based Access Control (RBAC)

The system implements Role-Based Access Control (RBAC) using Spring Security. This ensures that users only have access to the functionalities they need based on their roles. For instance:

	Category	Endpoint / Action	ADMIN	PRODUCT_MGR	WAREHOUSE_MGR	CUSTOMER_SUPPORT	CUSTOMER
1	Users	GET /users	ALLOW	DENY	DENY	ALLOW	DENY
2		POST/PUT/DELETE /users	ALLOW	DENY	DENY	DENY	DENY
3							
4	Orders	GET /orders	ALLOW	DENY	ALLOW	ALLOW	DENY
5		GET /orders/{id}	ALLOW	DENY	ALLOW	ALLOW	ALLOW
6		POST /orders	ALLOW	DENY	DENY	DENY	ALLOW
7	Products	PUT /orders	ALLOW	DENY	ALLOW	DENY	DENY
8		GET /products (All/ID)	ALLOW	ALLOW	ALLOW	ALLOW	ALLOW
9		GET /products/my	ALLOW	ALLOW	ALLOW	DENY	DENY
10		GET /products/manager/{id}	ALLOW	DENY	ALLOW	DENY	DENY
11		POST /products	ALLOW	ALLOW	DENY	DENY	DENY
12		PUT /products/{id}	ALLOW	ALLOW	DENY	DENY	DENY
13	Inventory	DELETE /products/{id}	ALLOW	ALLOW	DENY	DENY	DENY
14		GET /inventory (All/ID/Low)	ALLOW	ALLOW	ALLOW	DENY	DENY
15		POST /inventory	ALLOW	ALLOW	ALLOW	DENY	DENY
16		PUT /inventory/{id}	ALLOW	ALLOW	ALLOW	DENY	DENY
17		DELETE /inventory/{id}	ALLOW	DENY	ALLOW	DENY	DENY
18							
18	Shipments	GET /shipments	ALLOW	DENY	ALLOW	ALLOW	DENY
19		POST/PUT /shipments	ALLOW	DENY	ALLOW	DENY	DENY
20	Returns	GET /returns	ALLOW	DENY	DENY	ALLOW	DENY
21		POST /returns	ALLOW	DENY	DENY	DENY	ALLOW

Table-2: Role Based Access Control

7. Security Considerations

Both roles are secured through robust authentication mechanisms provided by Spring Security. This includes:

- 7.1 Authentication:** Verifying the identity of users before granting access to the system.
- 7.2 Authorization:** Ensuring that users can only access the resources permitted by their role.
- 7.3 Token Management:** Secure handling of user sessions to prevent unauthorized access

8. Responsibilities:

Admin

- Create, manage, and delete user accounts.
- Modular architecture shall allow future extensions.
- Assign roles and permissions (Admin, Product Manager, Warehouse Manager, Customer Support, Customer).
- Monitor overall system usage and ensure smooth platform operation.
- Maintain master control over all modules and configurations.

Product Manager

- Add, update, and manage product details such as name, description, category, and pricing.
- Ensure product information consistency across the platform.
- Coordinate with Warehouse Managers to align product availability with stock data.
- Manage product lifecycle from creation to discontinuation.

Warehouse Manager

- Monitor product stock levels across warehouses.
- Update inventory quantities based on inbound and outbound movements.
- Coordinate warehouse operations for storage and dispatch.
- Ensure accurate warehouse and inventory data for order fulfilment.

Customer Support

- View customer orders and shipment details to assist customers.
- Handle customer queries, complaints, returns, and exchange requests.
- Track shipment status and provide real-time updates to customers.
- Act as an intermediary between customers and internal teams.

Customer

- Browse available products and view product details.
- Place orders and track shipment status in real time.
- Initiate return or replacement requests when applicable.
- View order history and manage personal account details.

9. Use Cases and User Stories

9.1. Amazon Warehouse Operations Scenario

In a large-scale e-commerce environment such as Amazon, daily warehouse operations depend on real-time inventory synchronization across multiple fulfilment centres.

The Warehouse Manager continuously updates stock levels to maintain accurate product availability. When a customer places an order, the system automatically deducts the ordered quantity from inventory, preventing overselling and enabling precise demand forecasting.

Once items are picked and packed, the system automatically generates a shipment and updates its status in real time. These updates are immediately visible to internal teams and customers, ensuring transparency throughout the order lifecycle. This level of automation minimizes manual intervention, reduces operational errors, and supports Amazon's fast and reliable delivery timelines.

9.2. FedEx Shipment Tracking Scenario

In the FedEx shipment tracking use case, the logistics system captures shipment events at every transit point. Each time a package arrives at, departs from, or is processed at a facility, an event is recorded in the system, such as:

- Arrived at Sorting Center
- Departed from Origin
- Out for Delivery

These events are updated in real time and made available to customers through the tracking interface. Live shipment visibility enhances transparency, reduces customer support inquiries, and builds trust by providing accurate delivery expectations. Internally, this data allows FedEx to optimize routing and proactively identify delays within the logistics chain.

9.3. Alibaba Bulk Inventory Management Scenario

In large-scale warehouses such as those operated by Alibaba, inventory management involves handling thousands of products simultaneously.

To efficiently manage over 1000+ products, the system provides:

- Pagination for large product datasets
- Advanced filtering and search mechanisms

These features enable warehouse staff to quickly locate products, update stock quantities, and manage bulk inventory without performance degradation, ensuring smooth warehouse operations even at high scale.

3.2 Use Case Diagram:

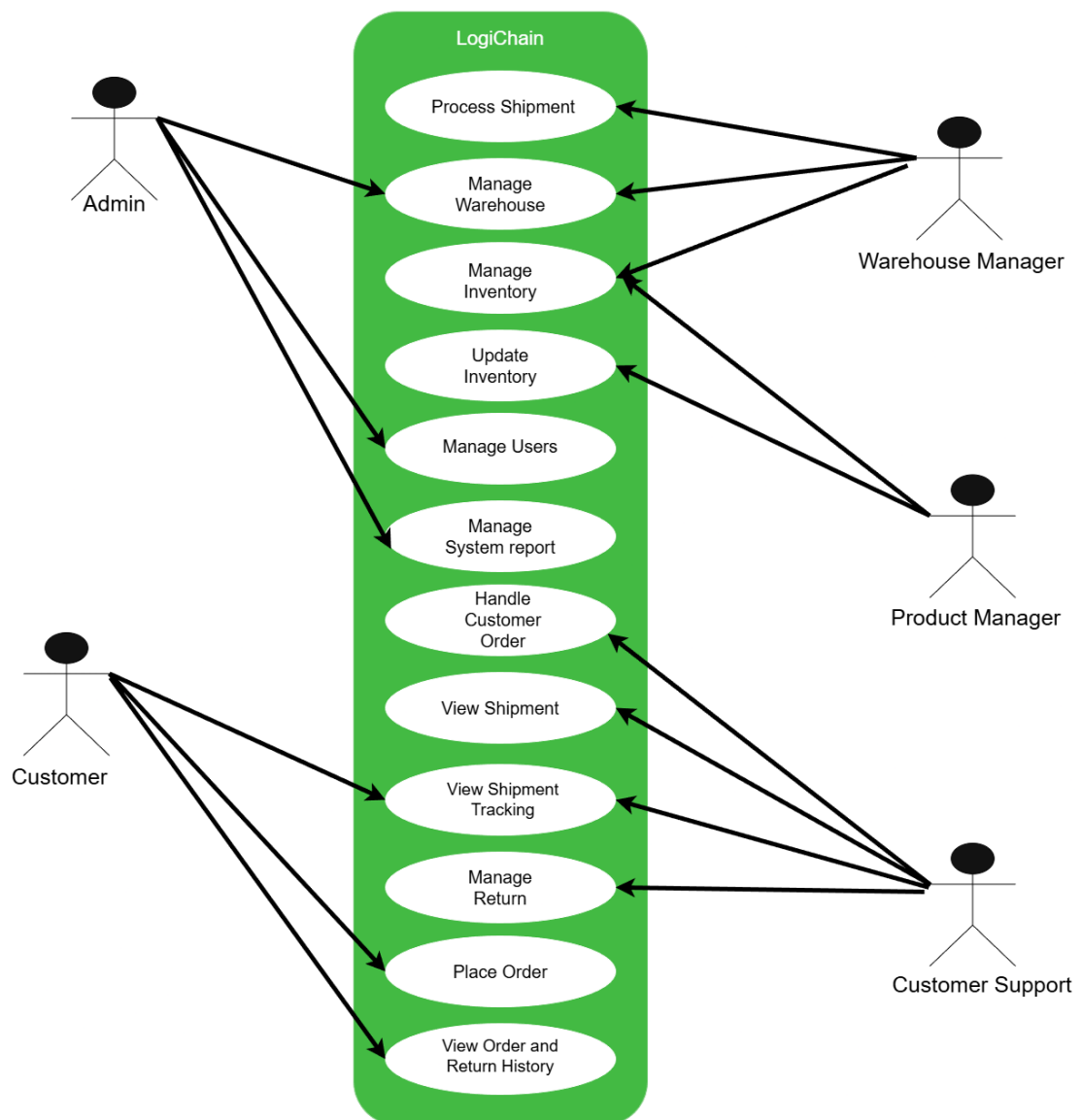
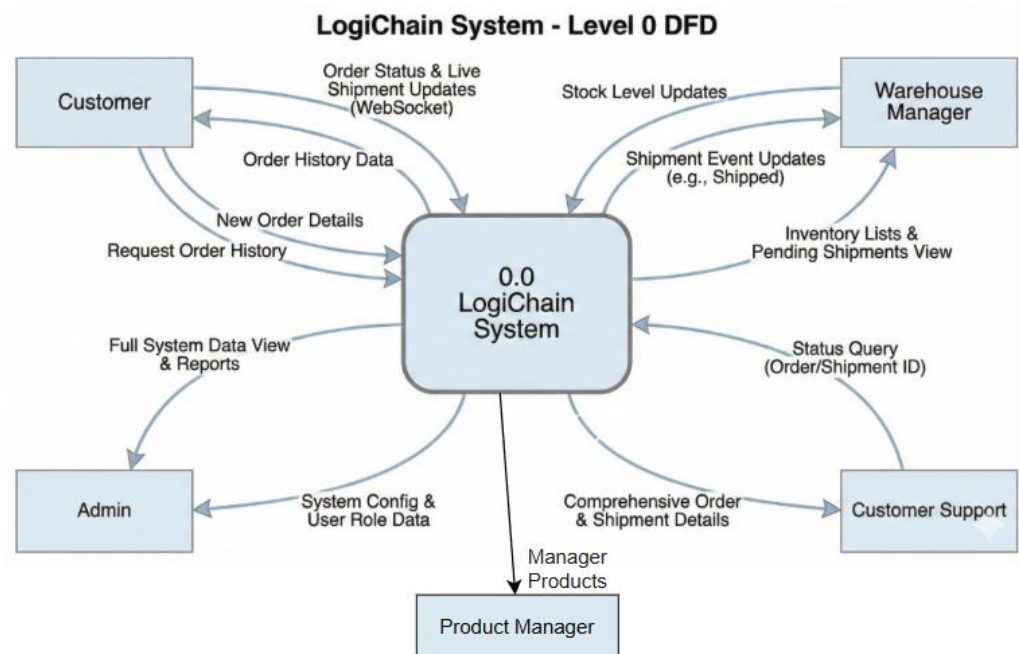


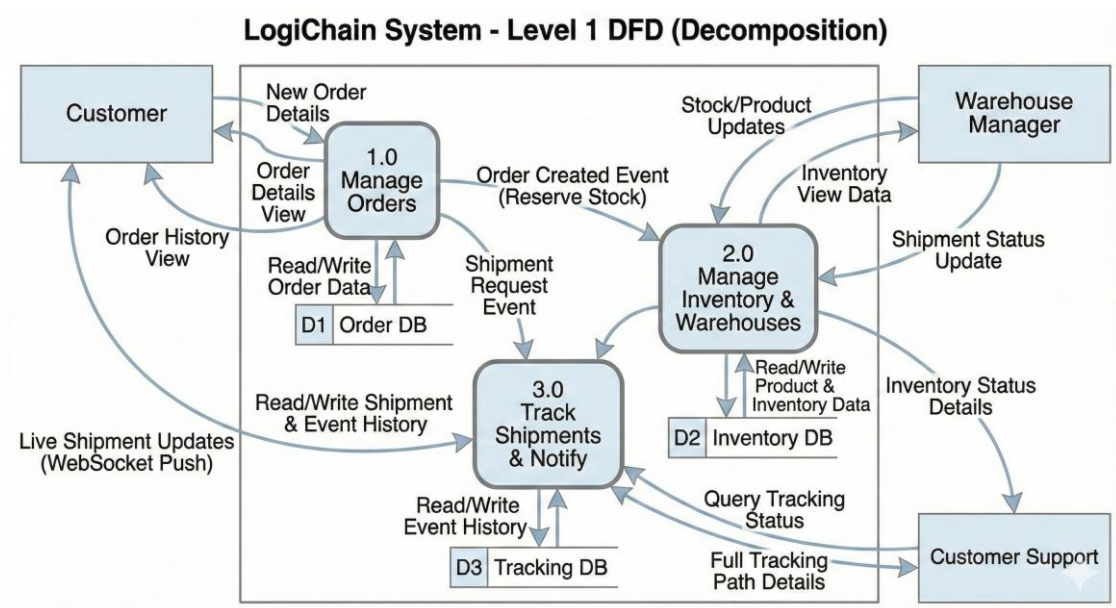
Fig. Use Case Diagram for LogiChain

3.3 Data Flow Diagram:

DFD Level 0:



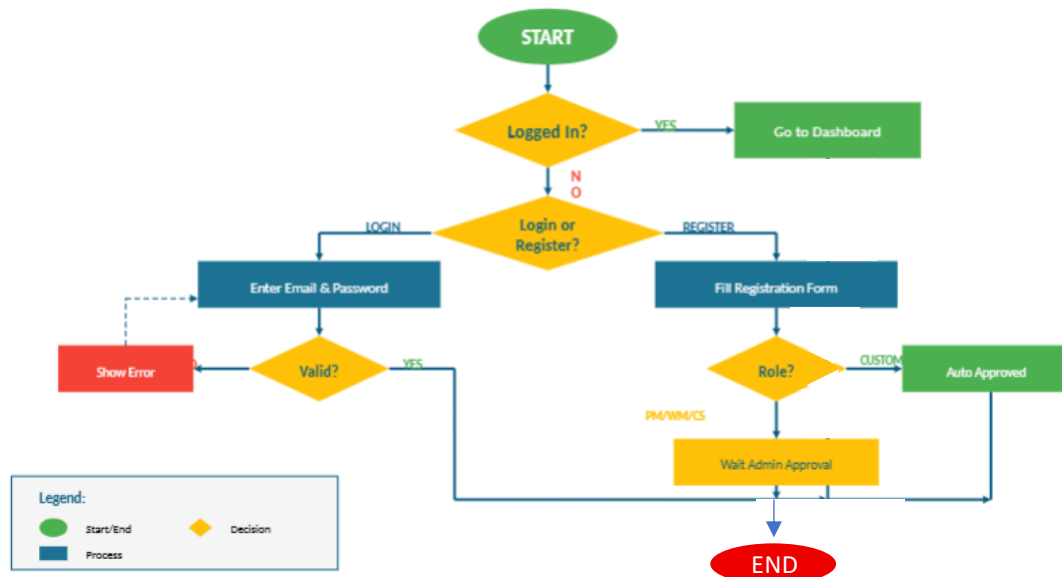
DFD level 1:



3.4 Activity Diagram:

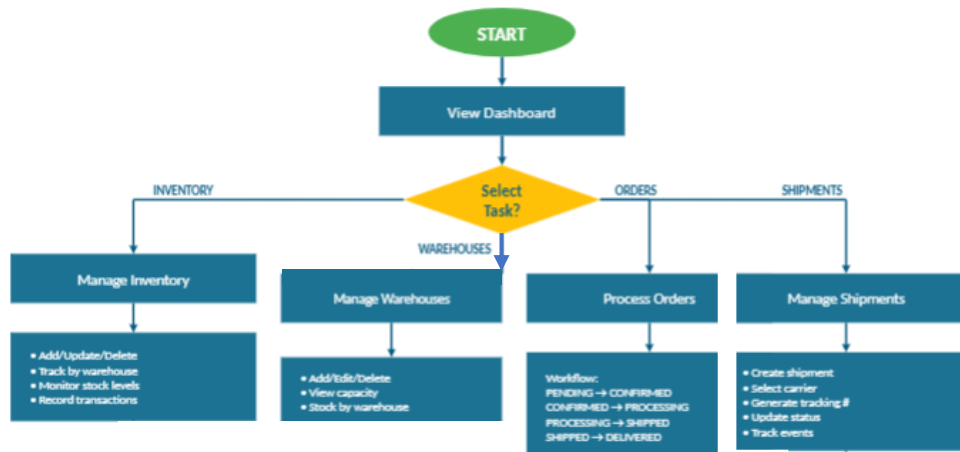
1. Login and Registration Activity Diagram

Login & Registration Activity Diagram

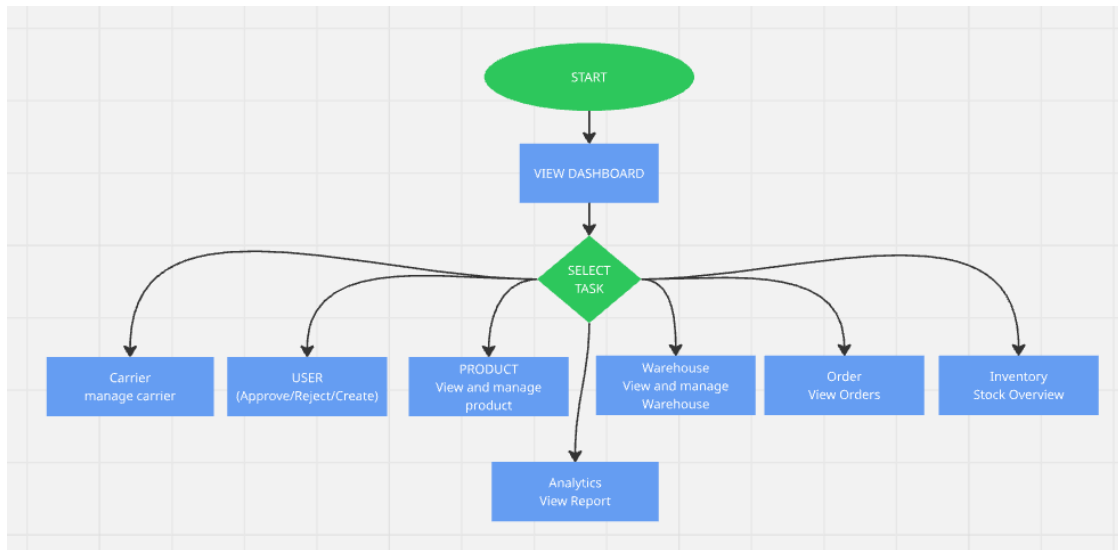


2. Warehouse Manager Activity Diagram

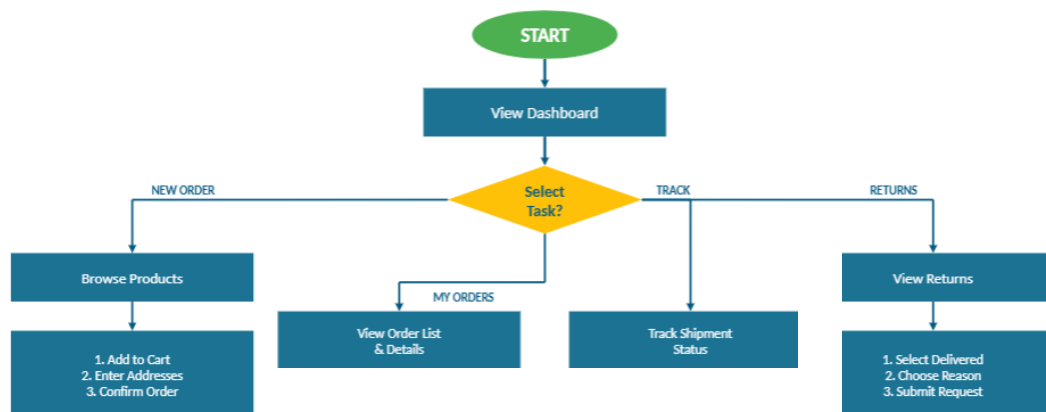
Warehouse Manager Activity Diagram



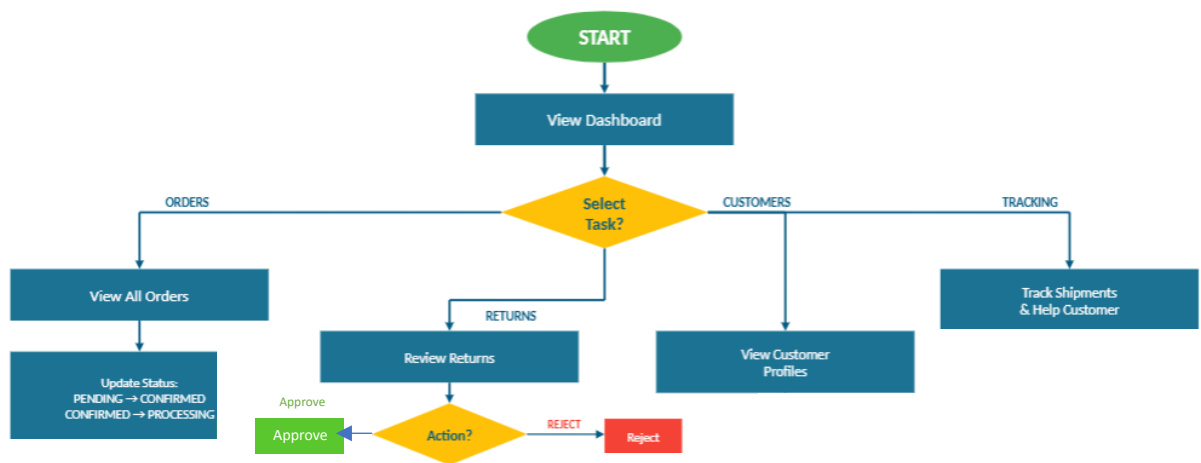
3. Admin Activity Diagram:



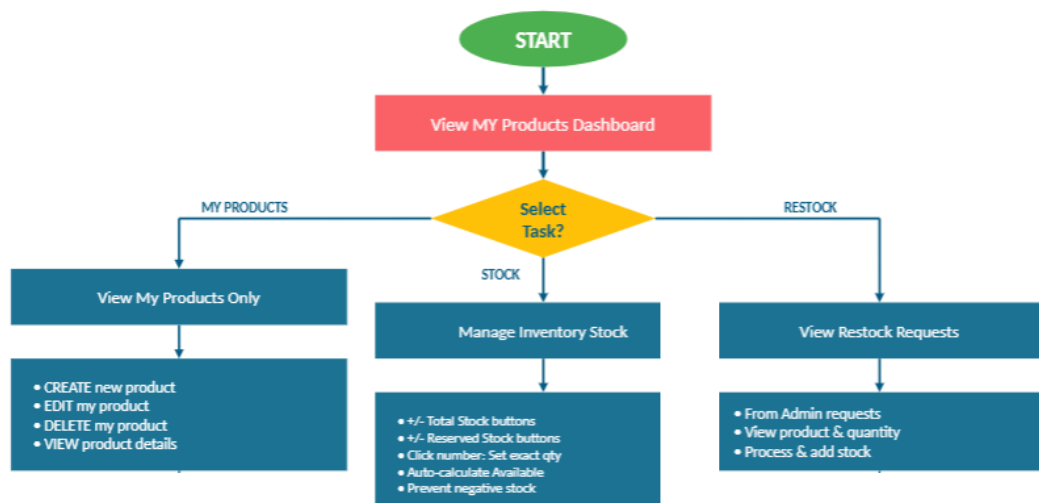
4. Customer Activity Diagram



5. Customer Support Activity Diagram



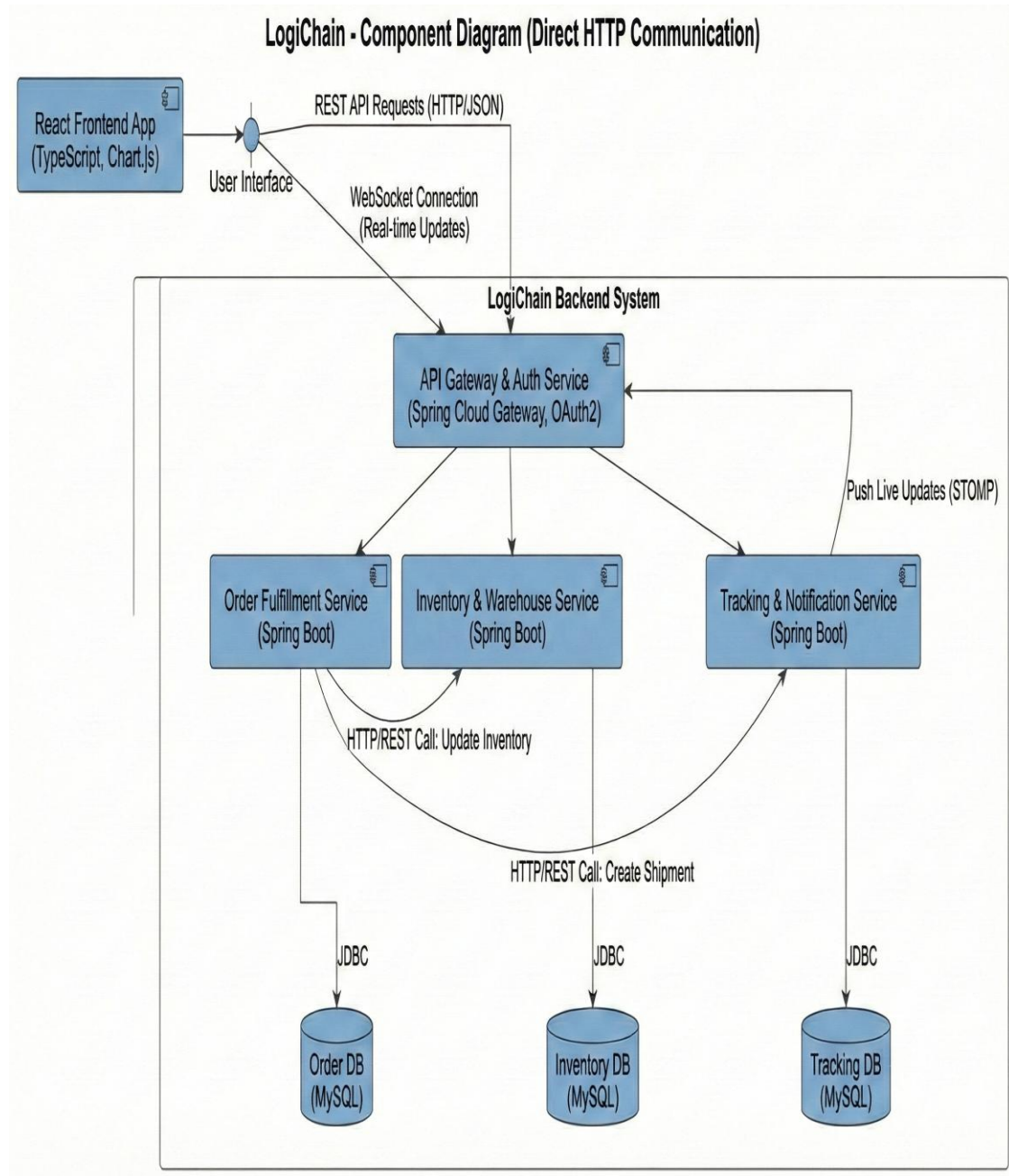
6. Product Manager Activity Diagram



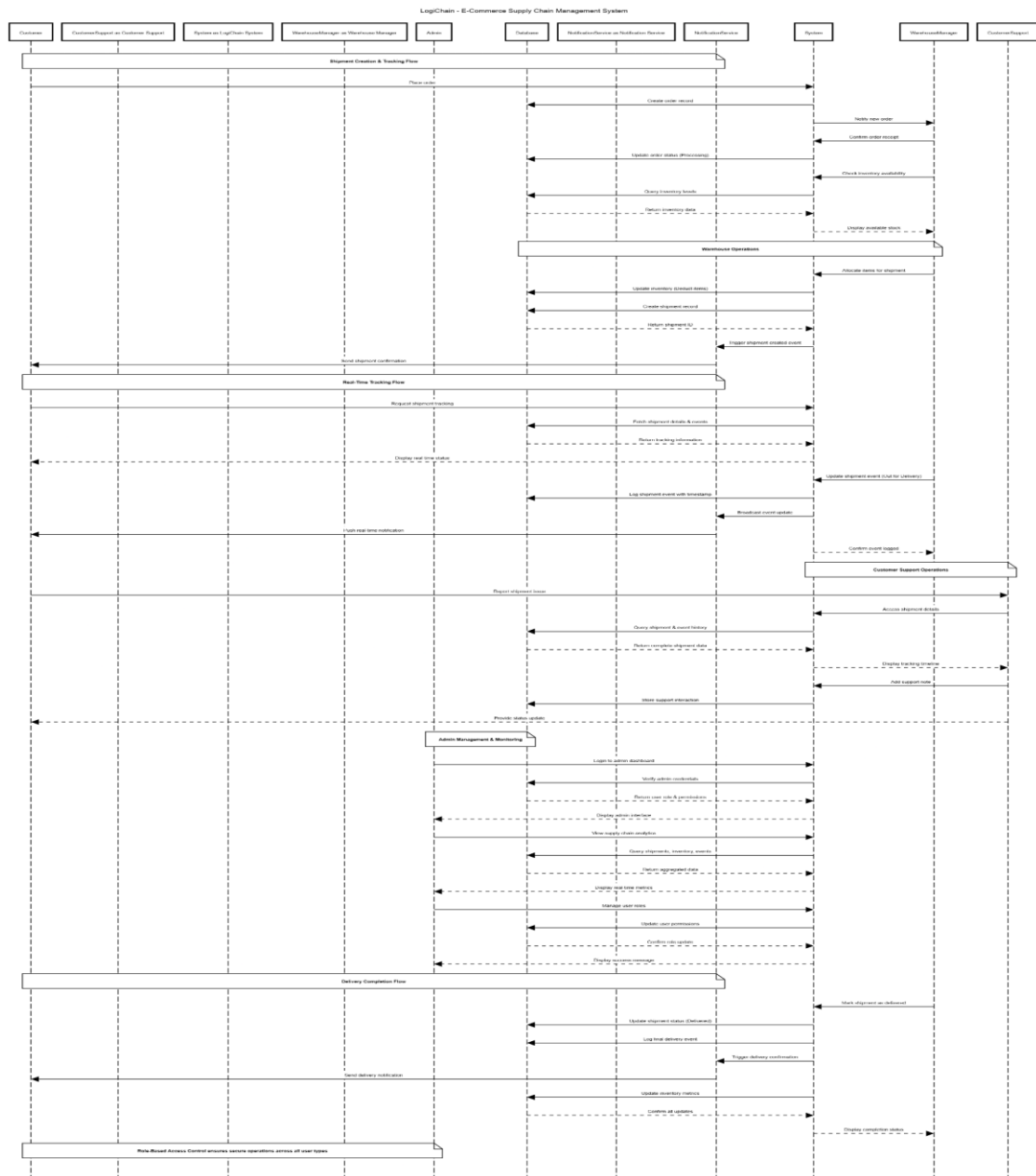
Inventory + Shipment System (Entities + Services + Controllers)



3.6 Component Diagram:

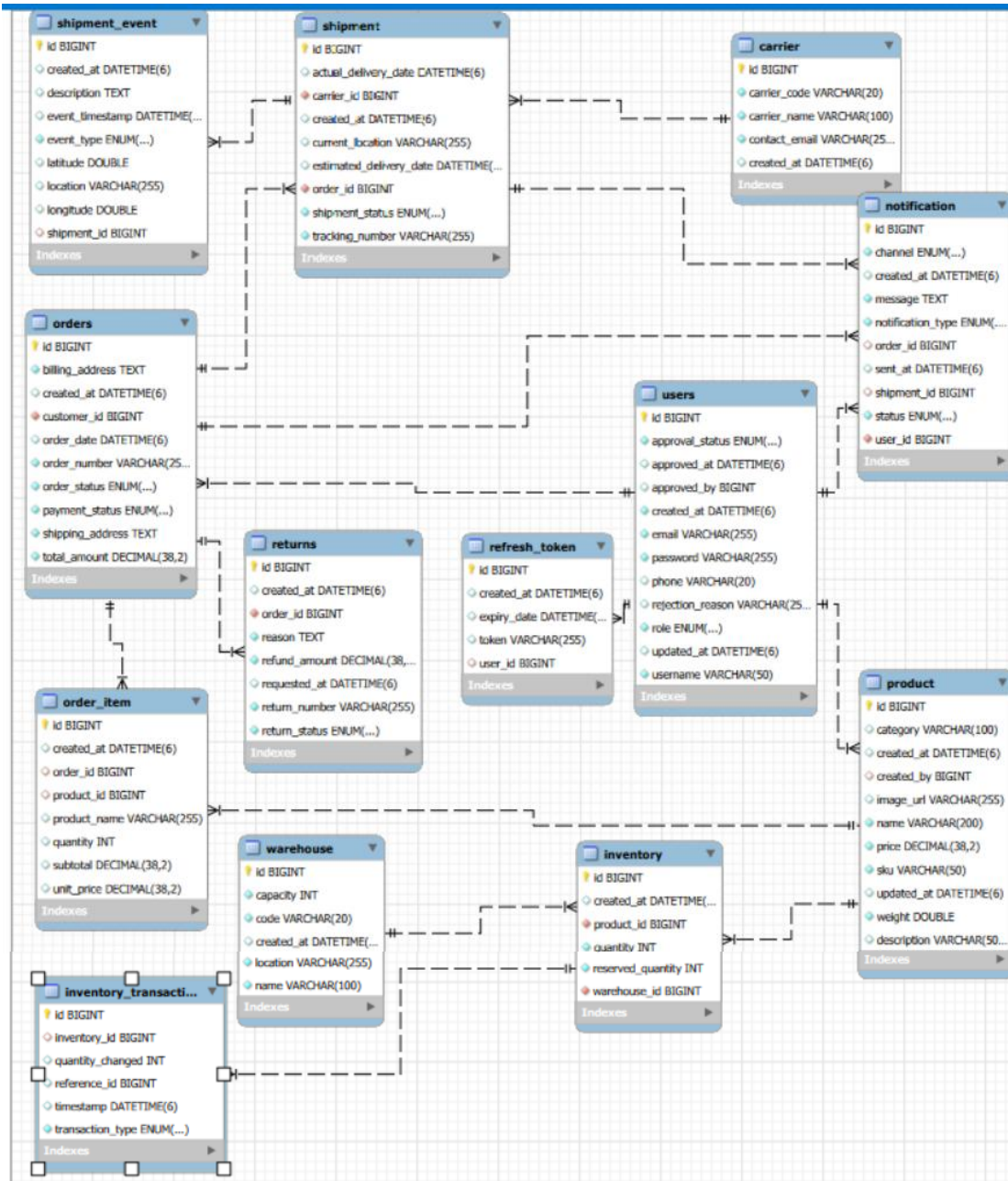


3.7 Sequence Diagram



4. DATABASE DESIGN

4.1 Design:



4.2 Tables:

The following table structures depict the database design.

```
mysql> desc users;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
approval_status	enum('APPROVED', 'PENDING', 'REJECTED')	NO		NULL	
approved_at	datetime(6)	YES		NULL	
approved_by	bigint	YES		NULL	
created_at	datetime(6)	NO		NULL	
email	varchar(255)	NO	UNI	NULL	
password	varchar(255)	NO		NULL	
phone	varchar(20)	YES		NULL	
rejection_reason	varchar(255)	YES		NULL	
role	enum('ADMIN', 'CUSTOMER', 'CUSTOMER_SUPPORT', 'PRODUCT_MANAGER', 'WAREHOUSE_MANAGER')	NO		NULL	
updated_at	datetime(6)	YES		NULL	
username	varchar(50)	NO	UNI	NULL	

12 rows in set (0.01 sec)

Table-3: users

```
mysql> desc product;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
category	varchar(100)	YES		NULL	
created_at	datetime(6)	YES		NULL	
created_by	bigint	YES	MUL	NULL	
image_url	varchar(255)	YES		NULL	
name	varchar(200)	NO		NULL	
price	decimal(38,2)	NO		NULL	
sku	varchar(50)	NO	UNI	NULL	
updated_at	datetime(6)	YES		NULL	
weight	double	NO		NULL	
description	varchar(500)	YES		NULL	

11 rows in set (0.00 sec)

Table-4: product

```
mysql> desc carrier;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
carrier_code	varchar(20)	NO	UNI	NULL	
carrier_name	varchar(100)	NO		NULL	
contact_email	varchar(255)	NO		NULL	
created_at	datetime(6)	YES		NULL	

5 rows in set (0.00 sec)

Table-5: carrier


```
mysql> desc inventory
-> ;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
created_at	datetime(6)	YES		NULL	
product_id	bigint	NO	MUL	NULL	
quantity	int	NO		NULL	
reserved_quantity	int	NO		NULL	
warehouse_id	bigint	NO	MUL	NULL	

6 rows in set (0.00 sec)

Table-6: inventory

```
mysql> desc inventory_transaction;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
inventory_id	bigint	YES	MUL	NULL	
quantity_changed	int	YES		NULL	
reference_id	bigint	YES		NULL	
timestamp	datetime(6)	YES		NULL	
transaction_type	enum('RELEASED', 'RESERVED', 'STOCK_IN', 'STOCK_OUT')	NO		NULL	

6 rows in set (0.00 sec)

Table-7: inventory_transactions

```
mysql> desc notification;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
channel	enum('EMAIL', 'PUSH', 'SMS')	NO		NULL	
created_at	datetime(6)	YES		NULL	
message	text	NO		NULL	
notification_type	enum('DELIVERED', 'ORDER_CANCELLED', 'ORDER_CONFIRMED', 'ORDER_PLACED', 'ORDER_PROCESSING', 'OUT_FOR_DELIVERY', 'PAYMENT_FAILED', 'PAYMENT_RECEIVED', 'REFUND_PROCESSED', 'RETURN_APPROVED', 'RETURN_REJECTED', 'RETURN_REQUESTED', 'SHIPPED')	NO		NULL	
order_id	bigint	YES	MUL	NULL	
sent_at	datetime(6)	YES		NULL	
shipment_id	bigint	YES	MUL	NULL	
status	enum('FAILED', 'PENDING', 'SENT')	NO		NULL	
user_id	bigint	NO	MUL	NULL	

10 rows in set (0.00 sec)

Table-8: notification

```
mysql> desc notification_template;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
body_template	text	YES		NULL	
created_at	datetime(6)	YES		NULL	
notification_type	enum('DELIVERED', 'ORDER_CANCELLED', 'ORDER_CONFIRMED', 'ORDER_PLACED', 'ORDER_PROCESSING', 'OUT_FOR_DELIVERY', 'PAYMENT_FAILED', 'PAYMENT_RECEIVED', 'REFUND_PROCESSED', 'RETURN_APPROVED', 'RETURN_REJECTED', 'RETURN_REQUESTED', 'SHIPPED')	NO		NULL	
subject	varchar(255)	YES		NULL	
template_name	varchar(255)	YES	UNI	NULL	

6 rows in set (0.00 sec)

Table-9: notification_template

```
mysql> desc orders;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
billing_address	text	NO		NULL	
created_at	datetime(6)	YES		NULL	
customer_id	bigint	NO	MUL	NULL	
order_date	datetime(6)	YES		NULL	
order_number	varchar(255)	NO	UNI	NULL	
order_status	enum('CANCELLED', 'CONFIRMED', 'DELIVERED', 'PENDING', 'PROCESSING', 'SHIPPED')	NO		NULL	
payment_status	enum('FAILED', 'PAID', 'PENDING', 'REFUNDED')	NO		NULL	
shipping_address	text	NO		NULL	
total_amount	decimal(38,2)	NO		NULL	

10 rows in set (0.00 sec)

Table-10: orders

```
mysql> desc order_item;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
created_at	datetime(6)	YES		NULL	
order_id	bigint	YES	MUL	NULL	
product_id	bigint	YES	MUL	NULL	
product_name	varchar(255)	YES		NULL	
quantity	int	YES		NULL	
subtotal	decimal(38,2)	YES		NULL	
unit_price	decimal(38,2)	YES		NULL	

8 rows in set (0.00 sec)

Table -11: order item

```
mysql> desc returns;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
created_at	datetime(6)	YES		NULL	
order_id	bigint	NO	MUL	NULL	
reason	text	NO		NULL	
refund_amount	decimal(38,2)	NO		NULL	
requested_at	datetime(6)	YES		NULL	
return_number	varchar(255)	NO	UNI	NULL	
return_status	enum('APPROVED', 'RECEIVED', 'REFUNDED', 'REJECTED', 'REQUESTED')	NO		NULL	

8 rows in set (0.00 sec)

Table-12: returns

```
mysql> desc shipment_event;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
created_at	datetime(6)	YES		NULL	
description	text	YES		NULL	
event_timestamp	datetime(6)	YES		NULL	
event_type	enum('CREATED', 'DELIVERED', 'FAILED', 'IN_TRANSIT', 'OUT_FOR_DELIVERY', 'PICKED_UP', 'RETURNED')	NO		NULL	
latitude	double	YES		NULL	
location	varchar(255)	YES		NULL	
longitude	double	YES		NULL	
shipment_id	bigint	YES	MUL	NULL	

9 rows in set (0.00 sec)

Table 13: shipment_events

```
mysql> desc shipment;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
actual_delivery_date	datetime(6)	YES		NULL	
carrier_id	bigint	NO	MUL	NULL	
created_at	datetime(6)	YES		NULL	
current_location	varchar(255)	YES		NULL	
estimated_delivery_date	datetime(6)	YES		NULL	
order_id	bigint	NO	UNI	NULL	
shipment_status	enum('CREATED', 'DELIVERED', 'FAILED', 'IN_TRANSIT', 'OUT_FOR_DELIVERY')	NO		NULL	
tracking_number	varchar(255)	NO	UNI	NULL	

9 rows in set (0.00 sec)

Table 14: shipments

```
mysql> desc warehouse;
```

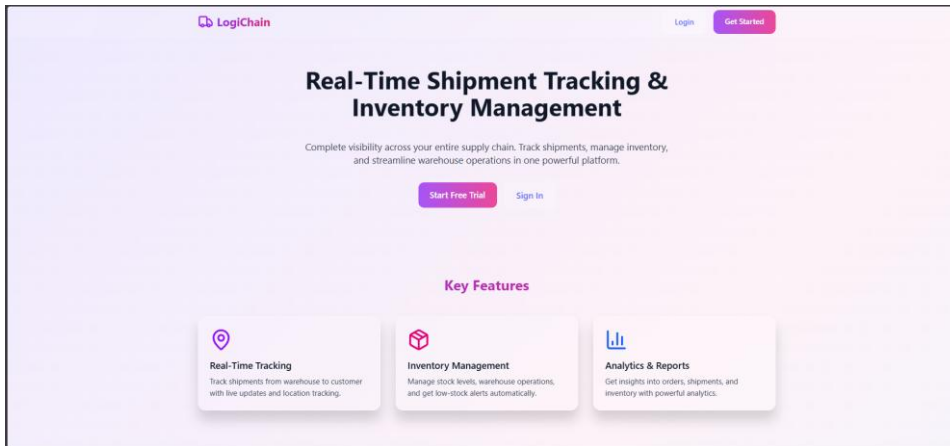
Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
capacity	int	NO		NULL	
code	varchar(20)	NO	UNI	NULL	
created_at	datetime(6)	YES		NULL	
location	varchar(255)	NO		NULL	
name	varchar(100)	NO		NULL	

6 rows in set (0.00 sec)

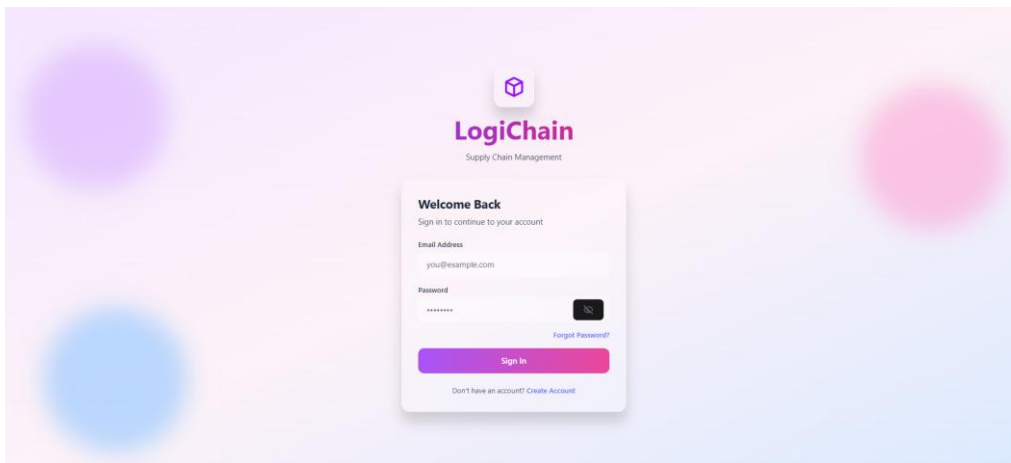
Table 15: warehouses

5. SNAPSHOTS

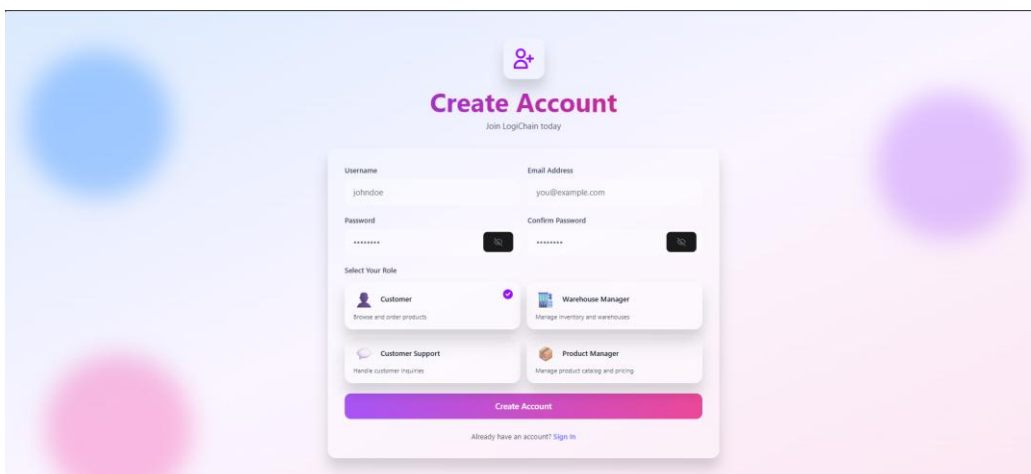
Getting Started Page:



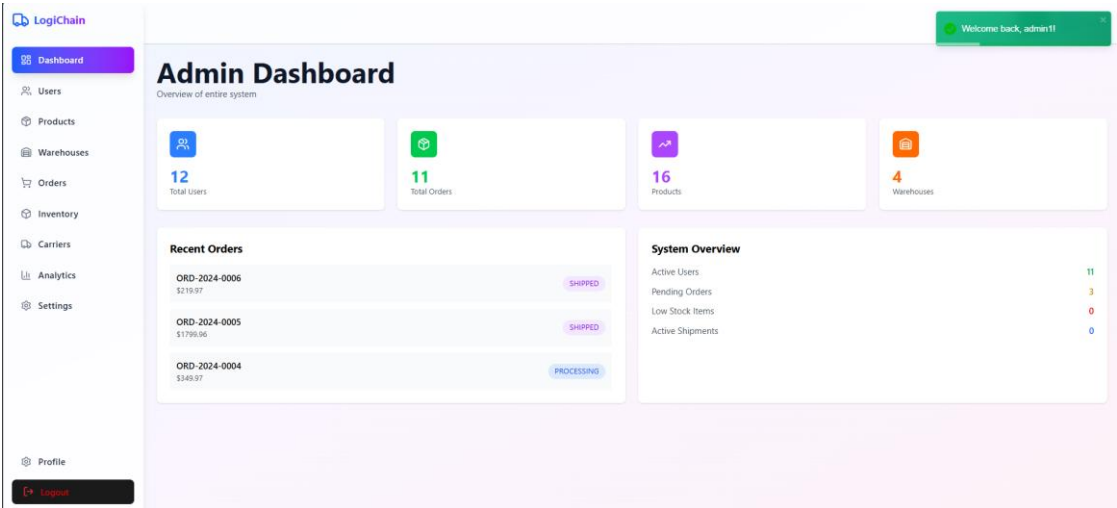
Login Page:



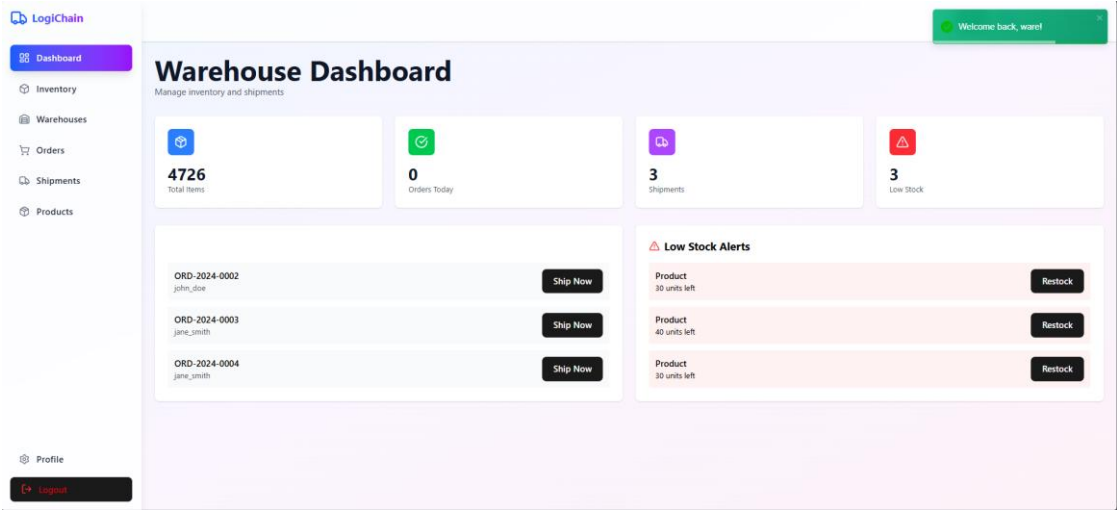
Registration Page:



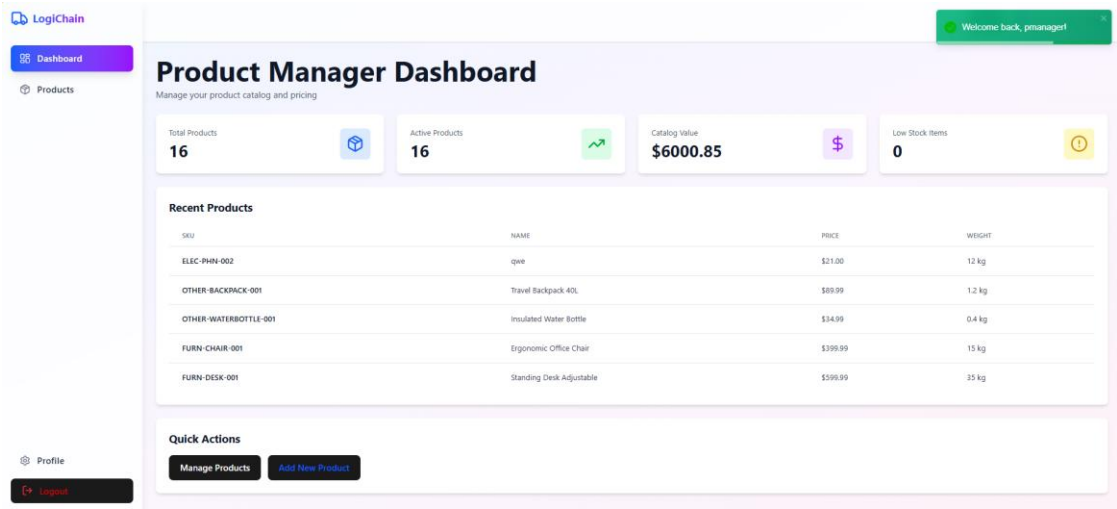
Admin Dashboard Page:



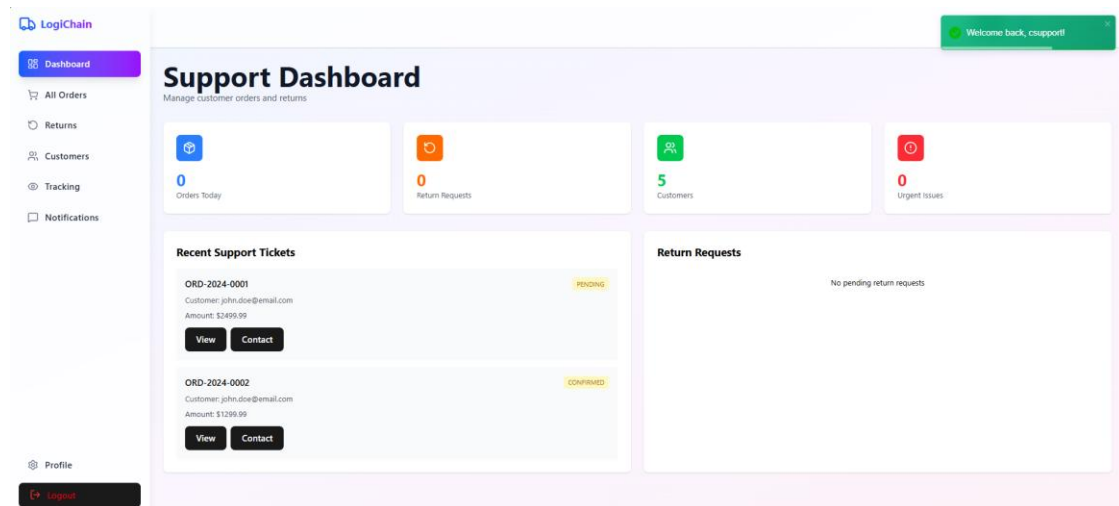
Warehouse Manager Dashboard Page:



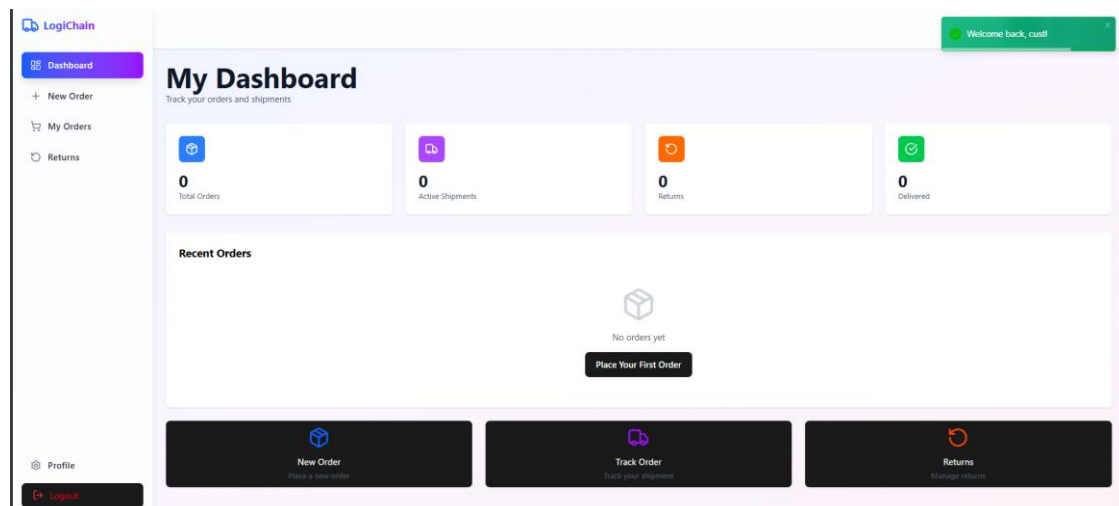
Product Manager Dashboard Page:



Customer Support Dashboard Page:



Customer Dashboard Page:



6. CONCLUSION

In conclusion, LogiChain successfully addresses the critical requirements of modern e-commerce supply chain management by providing real-time visibility into shipments, inventory, and order fulfilment processes. The system offers a structured and scalable solution that bridges the gap between warehouses, logistics operations, and customers, ensuring transparency and operational efficiency throughout the supply chain lifecycle.

The LogiChain project demonstrates a robust implementation of a real-time shipment and inventory tracking system using a microservices-based architecture. By leveraging Spring Boot, Spring Data JPA, and MySQL, the application ensures reliable data management, seamless service communication, and efficient handling of large transactional datasets. The inclusion of core entities such as Shipment, Inventory, Warehouse, Order, and ShipmentEvent enables precise tracking, auditing, and inventory control across multiple locations.

With features such as role-based authentication, customer access, secure password encryption using PasswordEncoder, and full CRUD operations, LogiChain prioritizes both security and usability. Pagination support enhances performance and user experience when managing large volumes of orders and shipments, while WebSocket-based live tracking enables near real-time status updates for shipments. Overall, LogiChain stands as a comprehensive and adaptable supply chain management solution. Its modular architecture, secure design, and use of modern web technologies provide a strong foundation for future enhancements such as advanced analytics, automated restocking, and third-party logistics integrations. The project is well-positioned to evolve alongside emerging technological trends and growing e-commerce demands, making it a valuable asset for businesses seeking efficient and transparent logistics operations.

7. REFERENCE

1. <https://docs.spring.io/spring-security/reference/>
2. <https://docs.spring.io/spring-boot/index.html/>
3. <https://www.amazon.com/Agile-Software-Development-Principles-Patterns/dp/0135974445>
4. Garcia-Molina, H., Ullman, J. D., & Widom, J. (2008). *Database Systems: The Complete Book*. Prentice Hall.
5. Codd, E. F. (1970). *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, 13(6), 377-387.
6. Fowler, M. (2003). *Patterns of Enterprise Application Architecture*. Addison- Wesley.
7. Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
8. Soni, D., Nord, R. L., & Hofmeister, C. (1995). *Software Architecture in Industrial Applications*. Proceedings of the 17th International Conference on Software Engineering, 196-207.